CHEN XINGQIANG

杭州，中国

+86 150-0133-4125 ◇ joy6677@outlook.com ◇ github.com/chenxingqiang

求职意向

寻求**全职**职位：**AI 加速器编译器 / 系统软件 · 大语言模型推理系统 · 软硬件协同设计**。

交叉学科背景

理论物理与生物物理训练，贯通计算化学、分子动力学与计算机科学。竞赛：**全国大学生数学建模竞赛国家一等奖**（2012）、**北美大学生数学建模竞赛（MCM）一等奖**（2013）；**CCF 开源 GPU 创新生态赛 GPU 算子优化挑战赛二等奖**（2025）。

教育背景

生物物理学与凝聚态物理博士研究生（结业，学位答辩中） 2014 年 9 月 - 2018 年 7 月
厦门大学，物理与机电工程学院；导师方向：蛋白质动力学与 QM/MM 仿真

理论物理学学士 2010 年 9 月 - 2014 年 7 月
厦门大学，物理与机电工程学院

技能与项目

编程语言：C++、Python（主要）；CUDA、Java、Scala、Go。

编译器 / 工具链：MLIR、LLVM 后端与 ISA 适配、TVM、图划分与算子融合、成本建模、CodeGen、自动调优。

芯片软件 / 验证：Palladium Emulation、GDB 与仿真模型联调、固件与权重映射生成、算子级指标（Cycles/MACs）分析。

推理系统：Transformers、KV-Cache、量化、批处理与调度、Triton 内核、FlashAttention、CUPTI/Perfetto 性能剖析。

平台：PyTorch、TensorFlow、ONNX、Kubernetes；Spark、Flink、Hadoop。

开源与发布（默认位于 `github.com/chenxingqiang`，均可公开验证）：

- **TraceSmith**：跨平台 GPU Profiling & Replay 系统，覆盖 CUDA/ROCm/Metal/MACA/Ascend；C++ 核心 + Python 绑定，已发布至 PyPI。`TraceSmith`
- **YiRage 持久化内核（YPK）**：LLM 超级内核编译器与运行时，多后端（CUDA/CPU/MPS/Ascend/Triton）。`YiRage`
- **JAX-sklearn**：scikit-learn 兼容接口 + JAX 加速，13,058 项测试通过，PyPI 发布。`jax-sklearn`
- **Secret-Learn**：隐私保护机器学习库，覆盖联邦学习／秘密分享／分割学习，PyPI 发布。`Secret-Learn`
- **YouTube-DNN-RecSys**：YouTube 风格召回与排序的 TensorFlow + Spark 实现（60 stars）。`YouTube-DNN-RecSys`
- **Research CLI**：面向科研工作流的命令行 AI 工具，多模型供应商与 MCP 集成。`iechor-research/research-cli`
- **Questions-Gen**：数学竞赛题目生成（Qwen3-14B、GRPO、蒸馏），含 CLI 与 Hugging Face 权重。`GenQuestion`

工作经历

编译器 & 工具链专家，知存算力（Witmem） 杭州，中国

编译器 / *Palladium* 验证 *2025* 年 *12* 月 *10* 日 - 至今

- 负责存算芯片产品线编译器与 Palladium 仿真验证，覆盖模型编译、指令生成、芯片执行及 Emulation 缺陷定位。
- 完成 Palladium 软件 Bring-up：Compiler 输出在仿真器上运行，结合 GDB 在仿真模型、芯片与系统软件层做联合调试。
- 实现 **CIM-MLIR** 异构编译器：PyTorch/TensorFlow/ONNX 经 TOSA/StableHLO 映射至 CPU、GPU 与 CIM，含前端、图划分、硬件后端与运行时。
- 实现代价驱动图划分与 CIMHigh/Mid/Low 三级 IR；用 OR-Tools 分配 Die/L2/L0 资源，生成可烧录 **config.bin**、权重 **map.bin**，以及 GPU PTX 与 LLVM 主机代码。

- 实现跨设备运行时：分设备执行器、统一内存、计算与传输重叠及三级缓冲；提供 C++/Python 接口及延迟、利用率、传输开销统计。
- 实现 **GRC-PIM**：PIM 上基于 drift gating 与 GEMV 复用的残差校准及软判决纠错，投稿 *npj Unconventional Computing*（详见发表论文）。

高级 AI 编译器工程师，亿铸智能
杭州，中国

研发部门 *2022* 年 *9* 月 - *2025* 年 *12* 月

- 主导 **YiRage 持久化内核（YPK）**：将端到端 LLM 解码编译为单个超级内核；在 Llama2-7B、单卡 A100 上相对 PyTorch eager 基线取得 **1.2×–6.7×** 端到端延迟收益（随 batch 与序列长度变化）。
- 实现融合算子（RMSNorm+Linear、GQA 等）与持久化执行模型，显著减少内核启动与中间张量往返；batch=1 decode 路径延迟降幅最大。
- 构建调度层并集成 Triton 与 FlashInfer 内核，支持多卡扩展（最高验证 8 卡）与 NVSHMEM 张量。
- 开发图编译与自动调优：内核参数搜索、KV-cache 友好内存布局，降低访存开销。
- 申请 **2 项**发明专利；相关能力交付 **2 个**数据中心客户 PoC。
- 负责公司 SoC 的 LLVM 后端 Target 与复杂指令集适配，推进编译器后端规范化与软件生态建设。
- 搭建模型转换与算子级评估流程：将 TVM、ONNX、MLCommons、TensorFlow/PyTorch 集成入 YZlite，统计 Cycles/MACs 等指标并输出多芯片 benchmark。开源工具包 `ReRAM-Labs/yzlite`。

隐私保护算法工程师，蓝象智联
杭州，中国

IT Group *2022* 年 *1* 月 - *2022* 年 *9* 月

- 构建基于同态加密的联邦学习平台，支撑多家金融机构的跨域建模；在保持隐私约束下相对本地基线提升模型效果，并缩短训练迭代周期。
- 设计多方安全计算协议，用于银行风控模型的联合训练与推理，兼顾协议安全性与通信开销。
- 开源 **Secret-Learn**：sklearn 兼容 API × 联邦学习／秘密分享／分割学习三种模式，JAX 加速，并与 SecretFlow 集成，PyPI 发布。`Secret-Learn`
- 申请 **3 项**发明专利（均已公开，可检索）：
- 基于秘密分享的纵向联邦线性支持向量分类方法 `CN114925786A`；随机森林训练方法及装置 `CN114997345A`；用于联邦学习的数据处理方法及装置 `CN115130681A`。

T7 算法工程专家，百图生科
北京，中国

算法工程组 *2021* 年 *9* 月 - *2021* 年 *12* 月

- 构建生物信息学工具链与数据平台，统一大规模生物数据的管理、检索与分析流程，支撑序列分析、结构预测与功能注释。
- 搭建蛋白质结构预测模型评估流程（含 AlphaFold 系列），为药物设计提供结构输入。
- 获 **2 项**软件著作权：生信工具自动化平台（2021SR2004464）、生物医药信息平台（2021SR1862397）；开源 AlphaFold2 代码解析与学习资源（26 stars）`alphafold2-codecs`。

高级算法工程师，Yalla 集团 (纽交所: YALA)
杭州，中国

数据中心工程组 *2020* 年 *9* 月 - *2021* 年 *9* 月

- 重构 Yalla 聊天室推荐系统，引入多目标排序；A/B 实验中人均日活跃时长与 7 日留存均取得显著正向（留存 **42%** → **61%**）。
- 构建 Ludo 风控系统，结合实时行为特征与机器学习模型识别异常账号，显著降低恶意退款比例。
- 开发卡牌类游戏 AI（深度强化学习），用于人机对局与匹配填充，提升低峰时段对局体验与留存。

高级算法工程师，格家网络科技
杭州，中国

搜索推荐工程组 *2019* 年 *3* 月 - *2020* 年 *9* 月

- 设计多场景推荐架构，融合协同过滤、深度排序模型与商品知识图谱；线上实验 CTR 与 CVR 均取得两位数相对提升。
- 优化实时推荐链路：特征缓存与召回裁剪，P99 响应控制在 **50ms** 内，支撑大促期间峰值流量。

机器学习工程师，便利蜂科技
北京，中国

数据工程组 *2017* 年 *3* 月 - *2019* 年 *3* 月

- 构建门店销量预测系统（覆盖 **50+** 门店），引入格点气象与时空特征，降低缺货与滞销率、提升库存周转。
- 开发用户与商品关联分析：在千万级行为数据上打通客户身份与购买记录，支撑个性化推荐。
- 设计商品视觉识别模型（**500+** SKU），用于快速结算场景，缩短收银与排队时间。

研究与实习

蛋白质分子动力学仿真与 QM/MM 模型
厦门大学，中国
(国家自然科学基金项目) *2014* 年 *9* 月 - *2017* 年 *3* 月

- 以 QM/MM 方法开展蛋白质动力学仿真与构象分析，成果发表于 Acta Cryst. D、PLoS ONE、Phys. Rev. E 等（见发表论文）。

液体活检 ct-DNA 试剂盒开发与 DNA 甲基化数据分析
乐普医疗，中国
(研发工程师实习) *2016* 年 *3* 月 - *2017* 年 *3* 月

- 在 TCGA/GEO 公开基因芯片数据上分析约 10,000 例样本，定位 Septin9 基因甲基化位点（结肠癌早期标志物），支撑临床诊断试剂盒的标志物筛选。

获奖与评审

- **国家一等奖**，全国大学生数学建模竞赛（CUMCM），2012 年。
- **一等奖**（Meritorious Winner），北美大学生数学建模竞赛（MCM/COMAP），2013 年。
- 全球**第 6 名**，国际太阳能十项全能竞赛（Solar Decathlon），2013 年。
- 第 **57** 名与第 **32** 名（共 1739 队），IEEE ICDM 全球气象 AI 挑战赛，2018 年。
- **二等奖**，第八届 CCF 开源创新大赛・开源 GPU 创新生态赛常规赛 S1 ・GPU 算子优化挑战赛（CCF/ODTC），2025 年。CCF 公示
- 评委，应用算法最佳实践初审，2021 年。

发表论文

- Chen X. GRC-PIM: A Drift-Gated, GEMV-Reusing Residual Calibrator for Soft-Decision Error Correction in Processing-in-Memory. *npj Unconventional Computing*（投稿中）。
- Cai Y, Su M, Ahmad A, Hu X, Sang J, Kong L, Chen X, Wang C, Shuai J, Han A. Conformational dynamics of the essential sensor histidine kinase WalK. *Acta Crystallographica Section D: Structural Biology*. 2017 Oct 1;73(10):793-803.
- Xia H, Chen X, Chen X, Cheng J, Liu Y, Chen X, Zhang Q, Zou G. Chiral discrimination of amino acid enantiomers based on different interactions with Cu2+. *Sensors and Actuators B: Chemical*. 2018 Jan 1;254:44-51.
- Ahmad A, Cai Y, Chen X, Shuai J, Han A. Conformational dynamics of response regulator RegX3 from Mycobacterium tuberculosis. *PloS one*. 2015 Jul 22;10(7):e0133389.
- Chen Y, Qi H, Li X, Cai M, Chen X, Liu W, Shuai J. Suppressing effect of C a 2+ blips on puff amplitudes by inhibiting channels to prevent recovery. *Physical Review E*. 2016 Aug 15;94(2):022411.

开源项目架构附录

陈星强 · github.com/chenxingqiang · 每页包含项目介绍与模块级系统架构

虚线框 = 架构分区
实线框 = 子系统
盒子 = 模块与职责

目录 CONTENTS · 每页一个项目

- TraceSmith — GPU 采集与回放
- YiRage — RuntimeFusion
- JAX-sklearn — 透明 JAX 加速
- Secret-Learn — 联邦 · 秘密分享 · 拆分
- YouTube-DNN — 候选召回
- Research CLI — Agent 工作流
- Questions-Gen — 三阶段训练

阅读约定

虚线框是架构分区，内层实线框是子系统，小盒子是具体模块。

每页说明系统包含哪些部分，以及各部分的职责边界。

TraceSmith — GPU Profiling 与 Replay 系统架构

应用入口、pybind11 绑定、C++17 核心库、GPU 后端与输出格式

虚线框 = 架构分区
实线框 = 子系统
盒子 = 模块与职责

项目介绍

TraceSmith 是面向 AI 编译器、深度学习框架与 GPU 驱动工程师的跨平台性能分析和确定性回放系统。

架构分为应用入口、pybind11 绑定、C++17 核心库、GPU 后端和输出格式。

APPLICATION · 用户入口

- **C++ Application** — 嵌入业务进程
- **Python Application** — 嵌入业务进程
- **C++ CLI** — tracesmith; profile · record · view
- **Python CLI** — tracesmith-cli; export · analyze · replay
- **External UI** — Perfetto UI; Chrome tracing

BINDINGS · Python / C++ 边界

- **_tracesmith.so** — python/src/bindings.cpp; pybind11 native extension

CORE LIBRARY · C++17

Cluster Module

- **GPUTopology** — NVLink · PCIe · NVML
- **MultiGPUProfiler** — 跨设备采集
- **TimeSync** — NTP / PTP / CUDA
- **NCCLTracker** — 集合通信
- **ClockCorrelator** — 时钟对齐
- **CommAnalysis** — 通信矩阵

Capture Module

- **CUPTIProfiler** — NVIDIA CUDA
- **MCPTIProfiler** — MetaX MACA
- **MetalProfiler** — Apple Metal
- **MemoryProfiler** — GPU 内存
- **AscendProfiler** — Huawei CANN
- **createProfiler()** — nsys · mcTracer; xctrace · msprof

State Module

- **GPUStateMachine** — 状态分析
- **InstructionStream** — 依赖图
- **TimelineBuilder** — 事件排序
- **TimelineViewer** — ASCII 渲染
- **PerfettoExporter** — JSON
- **PerfettoProtoExporter** — Protobuf

Replay Module

- **ReplayEngine** — 回放编排
- **StreamScheduler** — 操作顺序
- **OperationExecutor** — 执行回放
- **DeterminismChecker** — 结果比对
- **FrameCapture** — RenderDoc-style

Common Module

- **Types** — TraceEvent · EventType; KernelParams
- **RingBuffer** — lock-free SPSC
- **StackCapture** — libunwind
- **XRayImporter** — LLVM XRay
- **Timestamps** — ns precision
- **TraceMetadata** — DeviceInfo

Format Module

- **SBTWriter** — 二进制写入
- **SBTReader** — 二进制读取
- **.sbt** — header + events; counters

Configuration

- **ProfilerConfig** — 缓冲与溢出
- **ReplayConfig** — 模式与速度
- **StackCaptureConfig** — 调用栈策略
- **NCCLTrackerConfig** — 集合通信策略

GDB Module

- **RSPHandler** — GDB protocol
- **ProcessController** — ptrace / Mach
- **GPUDebugEngine** — 断点与回放
- **tracesmith-gdbserver** — v0.10.0

GPU BACKENDS · 系统接口

- **NVIDIA CUDA** — CUPTI SDK
- **Apple Metal** — Metal API
- **MetaX MACA** — MCPTI SDK
- **NCCL** — 分布式
- **Nsight Systems** — nsys
- **mcTracer** — MetaX
- **xctrace** — Apple
- **libunwind** — 主机调用栈
- **Perfetto SDK** — 轨迹导出

OUTPUTS · Trace 产物

- **trace.sbt** — 二进制格式
- **trace.json** — Perfetto JSON
- **trace.perfetto** — Protobuf
- **graph.dot** — Graphviz
- **ASCII Timeline** — 终端
- **Comm Matrix** — 热力图 JSON

C++17 核心包含采集、状态、格式、回放、集群、配置和 GDB。应用、绑定、后端和输出格式是它的边界。

YiRage (YPK) — RuntimeFusion 系统架构

推理引擎、RuntimeFusion、搜索计划、后端管理器、后端实现与硬件目标

虚线框 = 架构分区
实线框 = 子系统
盒子 = 模块与职责

项目介绍

YiRage 是面向大模型推理的多后端算子编译与运行时系统，核心能力是计算图超优化和 RuntimeFusion。

架构分为推理引擎、RuntimeFusion、搜索计划、后端管理器、后端实现和硬件目标。

INFERENCE ENGINE · 调度与 KV 所有者

- **vLLM** — continuous batch · PagedAttention
- **SGLang** — RadixAttention · extend meta
- **Decode Metadata** — block_table · seq_lens · radix hit · SM budget · batch

RuntimeFusion · RF.step(inputs, meta)

- **RF.step** — select 0..N capsules per decode step
- **FusionCapsule** — MLP gate_up + down · yirage.core
- **FusionCapsule** — segment pipeline · gate_up + down
- **Native Fallback** — engine native operators

SEARCH & PLAN · 离线搜索与预热

- **superoptimize** — KN / TB graph search
- **FusionPlan** — μGraph · profile + fingerprint
- **MuGraph cache** — shape bucket restore
- **Ray Coordinator** — 可选分布式搜索
- **Serving Tiers** — seed_verify · full_tb · full_tb_ray

PYTHON API 与 BACKEND MANAGER

- **KNGraph / TBGraph** — kernel transpiler
- **BackendRegistry** — search + execute · same backend
- **yirage.core** — Cython runtime
- **yirage.serving** — RF · Capsule · cert
- **MACA Config** — warp = 64 · mxcc
- **CPU MLIR JIT** — 可选

BACKEND IMPLEMENTATIONS · 后端实现

- **CUDA** — nvcc · warp 32
- **MACA** — mxcc · warp 64
- **CPU** — OpenMP · MKL
- **MPS** — Apple GPU
- **Ascend** — CANN
- **ROCm** — AMD GPU
- **Triton** — DSL kernels
- **+ extra** — TPU · XPU · FPGA · MLIR

HARDWARE TARGETS · 硬件目标

- **NVIDIA GPU**
- **MetaX C500/C550**
- **x86 / ARM CPU**
- **Apple Silicon**
- **Huawei Ascend**
- **AWS Neuron** — Intel XPU

推理引擎拥有调度与 KV。RuntimeFusion 拥有 FusionCapsule。BackendRegistry 将搜索和执行绑定到同一后端。

JAX-sklearn (xlearn) — 透明加速系统架构

用户 API、兼容层、JAX 实现、数据策略与硬件后端

虚线框 = 架构分区
实线框 = 子系统
盒子 = 模块与职责

项目介绍

JAX-sklearn 在保持 scikit-learn 调用接口不变的前提下，透明地以 JAX 加速受支持的估计器。

架构分为用户 API、兼容层、JAX 实现、数据策略和硬件后端。

USER CODE · sklearn 兼容接口

- **import xlearn as sklearn**
  100% call surface
- **Estimators**
  linear_model · cluster
  decomposition · preprocessing
- **Calls**
  fit · predict · transform

COMPATIBILITY · xlearn/__init__.py

- **__getattr__**
  惰性导入子模块
- **_auto_jax_accelerate_module**
  替换估计器类
- **AcceleratorRegistry**
  _accelerator.py
- **create_intelligent_proxy**
  _proxy.py
- **monkey_patch_estimator**
  安装代理类

JAX ACCELERATION · xlearn/_jax

- **EstimatorProxy**
  fit-time dispatch
- **create_accelerated_estimator**
  _accelerator.py
- **_universal_jax.py**
  JIT · vmap
- **original_class**
  sklearn 回退
- **_gp.py**
  Gaussian process
- **_mlp.py**
  neural net
- **_svm.py**
  SVM kernels
- **_tree.py**
  tree models

DATA MANAGEMENT · 数组与策略

- **_data_conversion.py**
  NumPy ↔ JAX
  dtype · shape
- **_config.py**
  enable_jax
  jax_platform
- **jax_auto_threshold**
  complexity >= 1e8
- **detect_hardware**
  install command
  platform probe

HARDWARE · JAX 后端

- **CPU**
  jax-cpu · jaxlib
- **CUDA GPU**
  jax-gpu
  jax-cuda11
- **TPU**
  jax-tpu
- **Apple Metal**
  jax-metal
  float32 · experimental

兼容层把 EstimatorProxy 放在 sklearn API 与 JAX 实现或原生估计器之间。

Secret-Learn — 隐私机器学习系统架构

应用场景、sklearn API、FL / SS / SL 模式、JAX 加速、安全设备与基础设施

虚线框 = 架构分区
实线框 = 子系统
盒子 = 模块与职责

项目介绍

Secret-Learn 以 sklearn 兼容接口提供联邦学习、秘密分享和拆分学习三种隐私机器学习模式。
架构包括 sklearn API、三种隐私模式、JAX 加速、SecretFlow 安全设备和基础设施。

APPLICATION · 多方使用场景

- **Healthcare** — 跨院协作 · 患者隐私
- **Finance** — 跨行分析 · 欺诈检测
- **IoT / Edge** — 边缘智能 · 分布式学习
- **Research** — 基因组 · 多机构
- **Government** — 普查分析 · 跨部门数据

SKLEARN API · 191 个算法

- **Linear Models** — 39 · Ridge · Lasso · SGD
- **Ensemble** — 18 · RF · XGBoost
- **Clustering** — 14 · KMeans · DBSCAN
- **Decomposition** — 14 · PCA · NMF · ICA
- **SVM** — 7 · SVC · SVR
- **Neural Networks** — 4 · MLP · BernoulliRBM
- **+ 95 More** — Neighbors · Trees · Naive Bayes

PRIVACY MODES · 执行位置

FEDERATED LEARNING · 联邦学习

- **Local PYU** — 数据留在本地
- **HEU Aggregate** — 安全聚合
- **FLLinearRegression** — federated_learning/
- **Horizontal FL** — 3–5×

SECRET SHARING · 秘密分享

- **SPU MPC** — 密文计算
- **ABY3 / CHEETAH** — MPC 协议
- **SSPCA** — secret_sharing/
- **Maximum Privacy** — 1–2×

SPLIT LEARNING · 拆分学习

- **Model Split** — 跨参与方切分
- **HEU Activations** — 保护中间激活
- **SLMLPClassifier** — split_learning/
- **Vertical FL** — 2–4×

JAX ACCELERATION · xlearn 桥接

- **JIT / XLA** — 跟踪执行
- **jax.vmap** — 批处理与自动微分
- **xlearn Backend** — JAX-sklearn 桥
- **Smart Fallback** — 回退原生 sklearn
- **CPU** — 1.5–2.5×
- **GPU** — 5–8×
- **TPU** — 9–15×
- **M1 / M2** — 2–4×

SECURE DEVICES · SecretFlow 设备

SPU

- **MPC** — ABY3 · CHEETAH
- **Ciphertext** — 不暴露明文

HEU

- **Schemes** — Paillier · CKKS
- **Aggregation** — 梯度保护

TEE

- **Enclaves** — Intel SGX · AMD SEV
- **Occlum** — 证明

PYU

- **Party Local** — 数据归属
- **JAX Local** — 本地明文计算

INFRASTRUCTURE · 运行时基础设施

- **SecretFlow** — Ray distributed · gRPC · DP-SGD
- **JAX / XLA** — CUDA / cuDNN · TPU runtime
- **sklearn** — NumPy · SciPy

FL、SS、SL 是同一 sklearn API 的三种部署形态。JAX 与 SecretFlow 的 PYU、HEU、SPU、TEE 构成计算边界。

YouTube-DNN-RecSys — 候选召回系统架构

数据处理、存储、dnn.py 模型、预测服务与辅助工具

虚线框 = 架构分区
实线框 = 子系统
盒子 = 模块与职责

项目介绍

YouTube-DNN-RecSys 使用 TensorFlow 与 Spark 实现 YouTube 推荐系统中的候选召回阶段。

架构分为数据处理、存储、dnn.py 模型、预测服务和辅助工具。

DATA PROCESSING · 数据处理层

Python 数据处理

data2tfrecords.py
生成训练数据

TFRecords
训练与评估集

read_tfrecords.py
数据集读取

Scala + Spark 数据处理

MakeDataOne.scala
分布式生成

MakeDataTwo.scala
TFRecords output

HDFS
数据集存储

STORAGE · 存储层

data/tfrecords/
train/train.tfrecords
evaluation/evaluation.tfrecords

data/checkpoints/
ckpt/checkpoint
ckpt/eval · modelpath/

MODEL TRAINING · dnn.py

Input
visit_items_index × 5
continuous_features × 16

Item Embedding
500 items × 16

Average Pooling
历史向量

DNN Tower
Dense 64 + ReLU
Dropout · Dense 32 + ReLU
user vector 16

Softmax Output
500 classes
cross entropy · Top-K

PREDICTION · 预测服务层

PredictUserVector.scala
SavedModel
user vector head

ItemEmbeddingPredictor.scala
item embedding table
FeatureBuilder

load_dnn_model.py
SavedModel load
在线与批量推理

UTILITIES · 工具与辅助层

tensor_board.py
TensorBoard

feature_column.py
特征列

self_defined_network_layer.py
自定义网络层

Spark Base
BaseSparkLocal.scala
BaseSparkOnline.scala

Configuration
vars.prod.properties
vars.sit.properties

模型包含输入、物品 embedding、平均池化、用户塔和 softmax。Python 与 Spark 各自提供数据和预测组件。

Research CLI — Agent 系统架构

CLI、核心会话与路由、工具和研究模块、模型供应商、嵌入运行时

虚线框 = 架构分区
实线框 = 子系统
盒子 = 模块与职责

项目介绍

Research CLI 是面向代码、工具、MCP 服务和学术研究工作的终端 Agent。

架构分为 CLI、核心会话与路由、工具和研究模块、模型供应商、嵌入运行时。

CLI FRONTEND · packages/cli

- **research.tsx** — 交互终端
- **commands/** — mcp.ts · skills.tsx · hooks.tsx
- **ui/commands/research** — arxivCommand · paper · investigate · submit
- **acpSession.ts** — ACP session · stdio transport

CORE ORCHESTRATION · packages/core

会话

- **researchChat.ts** — researchRequest.ts
- **research-config-manager.ts** — 学术配置
- **config / memory** — 上下文

路由

- **modelRouterService** — routing/modelRouterService.ts
- **defaultStrategy** — routingStrategy.ts
- **classifierStrategy** — gemma · numerical
- **compositeStrategy** — approval · fallback · override

TOOLS 与 RESEARCH MODULES

Workspace

- **edit.ts · grep.ts** — filesystem changes
- **shell / ls** — 本地执行

MCP

- **mcp-client.ts** — 服务会话
- **mcp-client-manager.ts** — 资源路由

analysis/

- **bibliography-manager** — citations
- **experiment-code-generator** — Python · R · Julia
- **research-data-analyzer** — 统计与机器学习

bibliography/

- **arxiv-mcp-client** — arXiv search
- **google-scholar-client** — scholar search
- **literature-investigator** — keyword-generator · reference-classifier

submission/

- **latex-manager** — project compile
- **journal-matcher** — 期刊匹配
- **arxiv-latex-extractor** — 模板提取

MODEL PROVIDERS · 模型供应商

- **Gemini / Vertex**
- **OpenAI**
- **Anthropic**
- **Baidu ERNIE**
- **DeepSeek**
- **Qwen**
- **Groq / Mistral** — Cohere · Moonshot

EMBEDDED RUNTIMES · 嵌入运行时

- **packages/sdk** — 库嵌入
- **packages/a2a-server** — agent executor · HTTP task API
- **vscode-ide-companion** — IDE companion
- **packages/devtools** — 开发工具

CLI 与 ACP 构成前端。核心负责会话和模型路由。工具、研究模块、供应商和嵌入运行时是各自独立的边界。

Questions-Gen — 三阶段训练系统架构

训练语料、SFT、GRPO、蒸馏、验证与发布接口

虚线框 = 架构分区
实线框 = 子系统
盒子 = 模块与职责

项目介绍

Questions-Gen 是基于 Qwen3-14B 的数学竞赛题生成与评估系统。
架构包含训练语料、SFT、GRPO、蒸馏、验证和发布接口。

TRAINING DATA · 训练数据混合

- **Historical** — 50% — QuestionDataPreparer.py
- **Conditional Variants** — 30% — data_preparer.py
- **Innovative Types** — 20% — real_math_data_processor.py

Stage 1 · SFT · 监督微调

- **Qwen3-14B** — 基座模型
- **QuestionsGenTrainer.py** — src/ + questions_gen/core/trainer.py
- **stage1-fp-merged** — Hugging Face

Stage 2 · GRPO · 奖励优化

RewardCalculator.py

- **Difficulty** — 0.4 difficulty
- **Novelty** — 0.3
- **Rigor** — 0.2
- **Diversity** — 0.1

优化

- **GRPO Groups** — 8 problems/group
- **NoveltyConstraint.py** — 重复约束
- **stage2-merged-16bit** — variation generation

Stage 3 · Distill · 知识蒸馏

- **DeepSeekTeacher.py** — DeepSeek-R1
- **Teacher Signals** — 难度与严谨性 / 创新与教学价值
- **final-merged-16bit** — Hugging Face

VALIDATION · questions_gen/validation

- **model_validator.py** — 单模型
- **batch_validator.py** — algebra · calculus / geometry · number theory
- **quality_evaluator.py** — clarity · difficulty / originality

DEPLOYMENT · 发布与接口

- **main_cli.py** — validate · batch · quality
- **hf_utils.py** — three published stages
- **ollama_manager.py** — 本地推理
- **examples/** — quick_demo.py / demo_model_validation.py

SFT、GRPO 和蒸馏是三个模型阶段。RewardCalculator 与 NoveltyConstraint 属于 GRPO。验证和发布是独立接口。